TEKSTAC HANDS-ON

Spring Data JPA

01) School Strength

School.java

```
package com.bean;
import jakarta.persistence.*;
import java.util.ArrayList;
import java.util.List;
@Entity
@Table(name = "school")
public class School {
   @Id
   @Column(name = "school_id")
   private String schoolId;
   @Column(name = "school_name")
   private String schoolName;
   @Column(name = "school_type")
   private String schoolType;
   @Column(name = "city")
   private String city;
   @OneToMany(mappedBy = "school", cascade = CascadeType.ALL, fetch =
FetchType.LAZY)
   private List<Student> studentList = new ArrayList<>();
   public School() {}
   public String getSchoolId() {
      return schoolId;
   }
   public void setSchoolId(String schoolId) {
      this.schoolId = schoolId;
   }
   public String getSchoolName() {
      return schoolName;
   }
   public void setSchoolName(String schoolName) {
      this.schoolName = schoolName;
   }
   public String getSchoolType() {
```

```
        return schoolType;
    }
    public void setSchoolType(String schoolType) {
        this.schoolType = schoolType;
    }
    public String getCity() {
        return city;
    }
    public void setCity(String city) {
        this.city = city;
    }
    public List<Student> getStudentList() {
        return studentList;
    }
    public void setStudentList(List<Student> studentList) {
        this.studentList = studentList;
    }
}
```

Student.java

```
package com.bean;
import jakarta.persistence.*;
@Entity
@Table(name = "student")
public class Student {

    @Id
    @Column(name = "stud_roll_num")
    private String studentRollNumber;

    @Column(name = "student_name")
    private String studentName;

    @Column(name = "student_age")
    private int studentAge;

    @Column(name = "student_gender")
    private String studentGender;

    @Column(name = "student_grade")
    private String studentGrade;

    @Column(name = "student_section")
```

```
private String studentSection;

@ManyToOne
@JoinColumn(name = "schoolid")
private School school;

public Student() {}

public String getStudentRollNumber() {
   return studentRollNumber;
}

public void setStudentRollNumber(String studentRollNumber) {
   this.studentRollNumber = studentRollNumber;
}

public String getStudentName() {
   return studentName;
}

public void setStudentName(String studentName) {
   this.studentName = studentName;
}

public int getStudentAge() {
   return studentAge;
}

public void setStudentAge(int studentAge) {
   this.studentAge = studentAge;
}

public String getStudentGender() {
   return studentGender;
}

public void setStudentGender(String studentGender) {
   this.studentGender = studentGender;
}

public String getStudentGrade() {
   return studentGrade;
}
```

```
    public void setStudentGrade(String studentGrade) {
        this.studentGrade = studentGrade;
    }

    public String getStudentSection() {
        return studentSection;
    }

    public void setStudentSection(String studentSection) {
        this.studentSection = studentSection;
    }

    public School getSchool() {
        return school;
    }

    public void setSchool(School school) {
        this.school = school;
    }
}
```

SchoolDAO.java

```
package com.dao;

import com.bean.School;
import com.bean.Student;
import com.repository.SchoolRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Component;
import org.springframework.transaction.annotation.Transactional;

import java.util.List;
import java.util.Optional;

@Component
public class SchoolDAO {

    @Autowired
    private SchoolRepository schoolRepository;

    public void addSchool(School school) {
```

```
        schoolRepository.save(school);
    }

    @Transactional
    public void registerStudentToSchool(String schoolId, List<Student> students) {
        Optional<School> optionalSchool = schoolRepository.findById(schoolId);
        if (optionalSchool.isPresent()) {
            School school = optionalSchool.get();
            for (Student student : students) {
                student.setSchool(school);
                school.getStudentList().add(student);
            }
            schoolRepository.save(school);
        }
    }

    @Transactional(readOnly = true)
    public List<School> schoolWithMaximumStudents(String city) {
        return schoolRepository.findSchoolsWithMostStudentsInCity(city);
    }
}
```

SchoolRepository.java

```
package com.repository;

import com.bean.School;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import org.springframework.stereotype.Repository;

import java.util.List;

@Repository
public interface SchoolRepository extends JpaRepository<School, String> {

    @Query("SELECT s FROM School s WHERE s.city = :city AND size(s.studentList) = " +
            "(SELECT MAX(size(s2.studentList)) FROM School s2 WHERE s2.city = :city)")
    List<School> findSchoolsWithMostStudentsInCity(String city);
}
```

StudentRepository.java

```
package com.repository;

import com.bean.Student;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

@Repository
public interface StudentRepository extends JpaRepository<Student, String> {
}
```

SchoolStrengthApplication.java

```
package com;

import org.springframework.boot.CommandLineRunner;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class SchoolStrengthApplication implements CommandLineRunner {

   public static void main(String[] args) {
      SpringApplication.run(SchoolStrengthApplication.class, args);
   }

   @Override
   public void run(String... args) throws Exception {
      System.out.println("Invoke the methods");
   }
}
```

02) Loan Lenders

Pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>

<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"

        xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">

        <modelVersion>4.0.0</modelVersion>
```

```
<parent>
	<groupId>org.springframework.boot</groupId>
	<artifactId>spring-boot-starter-parent</artifactId>
	<version>3.0.6</version>
	<relativePath/> <!-- lookup parent from repository -->
</parent>
<groupId>com</groupId>
<artifactId>LoanLender</artifactId>
<version>0.0.1-SNAPSHOT</version>
<name>LoanLender</name>
<description>LoanLender</description>
<properties>
	<java.version>17</java.version>
</properties>
<dependencies>
	<dependency>
		<groupId>org.springframework.restdocs</groupId>
		<artifactId>spring-restdocs-mockmvc</artifactId>
		<scope>test</scope>
	</dependency>
	<dependency>
		<groupId>org.springframework.boot</groupId>
		<artifactId>spring-boot-starter-web</artifactId>
	</dependency>
	<dependency>
		<groupId>org.springframework.boot</groupId>
		<artifactId>spring-boot-starter-data-jpa</artifactId>
	</dependency>
	<dependency>
```

```
    <groupId>mysql</groupId>
    <artifactId>mysql-connector-java</artifactId>
    <version>8.0.26</version>
    <scope>runtime</scope>
</dependency>
        <dependency>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-test</artifactId>
            <scope>test</scope>
        </dependency>
        <dependency>
            <groupId>junit</groupId>
            <artifactId>junit</artifactId>
            <version>4.12</version>
            <scope>test</scope>
        </dependency>
    </dependencies>
    <build>
        <plugins>
            <plugin>
                <groupId>org.springframework.boot</groupId>
                <artifactId>spring-boot-maven-plugin</artifactId>

            </plugin>
            <plugin>
        <groupId>org.codehaus.mojo</groupId>
        <artifactId>exec-maven-plugin</artifactId>
        <version>1.6.0</version>
        <configuration>
```

```
                <mainClass>com.LoanLenderApplication</mainClass>
            </configuration>
            <executions>
                <execution>
                    <id>run-selenium</id>
                    <phase>integration-test</phase>
                    <goals><goal>java</goal></goals>
                </execution>
            </executions>
        </plugin>
            </plugins>
    </build>
</project>
```

Customer.java

```
package com.bean;

import jakarta.persistence.*;

import java.util.ArrayList;

import java.util.List;


@Entity

@Table(name = "Customer")

public class Customer {

    @Id

    private long customerId ;

    private String customerName ;
```

```
	private long customerContactNumber ;
	@OneToMany(mappedBy = "customer", cascade = CascadeType.ALL)
private List<Loan> loanList=new ArrayList<Loan>();
	public long getCustomerId() {
		return customerId;
	}
	public void setCustomerId(long customerId) {
		this.customerId = customerId;
	}
	public String getCustomerName() {
		return customerName;
	}
	public void setCustomerName(String customerName) {
		this.customerName = customerName;
	}
	public long getCustomerContactNumber() {
		return customerContactNumber;
	}
	public void setCustomerContactNumber(long customerContactNumber) {
		this.customerContactNumber = customerContactNumber;
	}
	public List<Loan> getLoanList() {
```

```
		return loanList;

	}

	public void setLoanList(List<Loan> loanList) {

		this.loanList = loanList;

	}
}
```

Loan.java

```
package com.bean;
import jakarta.persistence.*;

@Entity
@Table(name = "Loan")
public class Loan {
	@Id
	private long loanNumber ;
	private  String loanType ;
	private  double loanAmount ;
	private  int repaymentYears ;
	private  double monthlyDue ;

	@ManyToOne
   @JoinColumn(name = "customer_id")
	private Customer customer;
	public long getLoanNumber() {
		return loanNumber;
	}
	public void setLoanNumber(long loanNumber) {
		this.loanNumber = loanNumber;
	}
	public String getLoanType() {
		return loanType;
	}
	public void setLoanType(String loanType) {
		this.loanType = loanType;
	}
	public double getLoanAmount() {
```

```
		return loanAmount;
	}
	public void setLoanAmount(double loanAmount) {
		this.loanAmount = loanAmount;
	}
	public int getRepaymentYears() {
		return repaymentYears;
	}
	public void setRepaymentYears(int repaymentYears) {
		this.repaymentYears = repaymentYears;
	}
	public double getMonthlyDue() {
		return monthlyDue;
	}
	public void setMonthlyDue(double monthlyDue) {
		this.monthlyDue = monthlyDue;
	}
	public Customer getCustomer() {
		return customer;
	}
	public void setCustomer(Customer customer) {
		this.customer = customer;
	}




}
```

BankDAO.java

```
package com.dao;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Repository;
import com.bean.Customer;
import com.bean.Loan;
import com.repository.CustomerRepository;
import com.repository.LoanRepository;
import java.util.*;

@Repository
public class BankDAO {
```

```
    @Autowired
    private CustomerRepository customerRepo;

    @Autowired
    private LoanRepository loanRepo;

        public void addCustomer(Customer customer) {
                customerRepo.save(customer);
        }
        public void allocateLoanToCustomer(long customerId, Loan loan) {
                Customer customer = customerRepo.findById(customerId).orElseThrow();
      loan.setCustomer(customer);
      loanRepo.save(loan);
        }
        public List<Loan> loanWithMaxDueMinYears(){
                double maxDue = loanRepo.findMaxMonthlyDue();
      int minYears = loanRepo.findMinRepaymentYears();
      return loanRepo.findByMonthlyDueAndRepaymentYears(maxDue, minYears);
        }

}
```

LoanLenderApplication.java

```
package com;

import org.springframework.boot.CommandLineRunner;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class LoanLenderApplication implements CommandLineRunner {

        public static void main(String[] args) {
                SpringApplication app = new
SpringApplication(LoanLenderApplication.class);
      app.run(args);
        }
         @Override
           public void run(String... args) throws Exception {
                System.out.println("Invoke the methods");
            }
}
```

CustomerRepository.java

```
package com.repository;

import org.springframework.data.jpa.repository.JpaRepository;
import com.bean.Customer;

public interface CustomerRepository extends JpaRepository<Customer, Long> {
}
```

LoanRepository.java

```
package com.repository;

import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import com.bean.Loan;
import java.util.List;

public interface LoanRepository extends JpaRepository<Loan, Long> {

    @Query("SELECT MAX(l.monthlyDue) FROM Loan l")
    double findMaxMonthlyDue();

    @Query("SELECT MIN(l.repaymentYears) FROM Loan l")
    int findMinRepaymentYears();

    List<Loan> findByMonthlyDueAndRepaymentYears(double monthlyDue, int
repaymentYears);
}
```

Application.properties

```
spring.application.name=LoanLender
server.port=8090
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
spring.datasource.url=jdbc:mysql://localhost:3306/test23
spring.datasource.username=root
spring.datasource.password=@Ibm12345

spring.jpa.generate-ddl=true
spring.jpa.hibernate.ddl-auto=create
```

```
# Show or not log for each SQL query
spring.jpa.show-sql=true

# Allows Hibernate to generate SQL optimized for a particular DBMS
# spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQL8Dialect
```

03) Sales Statistics

Seller.java

```
package com.bean;

import jakarta.persistence.*;
import java.util.ArrayList;
import java.util.List;

@Entity
@Table(name = "seller")
public class Seller {

    @Id
    @Column(name = "seller_id")
    private String sellerId;

    @Column(name = "seller_name")
    private String sellerName;

    @Column(name = "seller_contact_num")
    private String sellerContactNumber;

    @OneToMany(mappedBy = "seller", cascade = CascadeType.ALL)
    private List<Car> carList = new ArrayList<>();

    public String getSellerId() {
        return sellerId;
    }

    public void setSellerId(String sellerId) {
        this.sellerId = sellerId;
    }

    public String getSellerName() {
```

```
        return sellerName;
    }

    public void setSellerName(String sellerName) {
        this.sellerName = sellerName;
    }

    public String getSellerContactNumber() {
        return sellerContactNumber;
    }

    public void setSellerContactNumber(String sellerContactNumber) {
        this.sellerContactNumber = sellerContactNumber;
    }

    public List<Car> getCarList() {
        return carList;
    }

    public void setCarList(List<Car> carList) {
        this.carList = carList;
    }
}
```

Car.java

```
package com.bean;

import jakarta.persistence.*;

@Entity
@Table(name = "car")
public class Car {

    @Id
    @Column(name = "reg_number")
    private String regNumber;

    @Column(name = "car_name")
    private String carName;

    @Column(name = "car_manufacturer")
    private String carManufacturer;
```

```
@Column(name = "car_model")
private String carModel;

@Column(name = "car_registration_date")
private String carRegistrationDate;

@Column(name = "car_price")
private double carPrice;

@ManyToOne
@JoinColumn(name = "sellerid")
private Seller seller;

public String getRegNumber() {
   return regNumber;
}

public void setRegNumber(String regNumber) {
   this.regNumber = regNumber;
}

public String getCarName() {
   return carName;
}

public void setCarName(String carName) {
   this.carName = carName;
}

public String getCarManufacturer() {
   return carManufacturer;
}

public void setCarManufacturer(String carManufacturer) {
   this.carManufacturer = carManufacturer;
}

public String getCarModel() {
   return carModel;
}

public void setCarModel(String carModel) {
```

```
        this.carModel = carModel;
    }

    public String getCarRegistrationDate() {
        return carRegistrationDate;
    }

    public void setCarRegistrationDate(String carRegistrationDate) {
        this.carRegistrationDate = carRegistrationDate;
    }

    public double getCarPrice() {
        return carPrice;
    }

    public void setCarPrice(double carPrice) {
        this.carPrice = carPrice;
    }

    public Seller getSeller() {
        return seller;
    }

    public void setSeller(Seller seller) {
        this.seller = seller;
    }
}
```

SellerRepository.java

```
package com.repository;

import com.bean.Seller;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import java.util.List;

public interface SellerRepository extends JpaRepository<Seller, String> {

    @Query("SELECT s FROM Seller s WHERE SIZE(s.carList) = " +
            "(SELECT MAX(SIZE(s2.carList)) FROM Seller s2)")
    List<Seller> sellerWithMaximumSalesCount();
}
```

CarRepository.java

```
package com.repository;

import com.bean.Car;
import org.springframework.data.jpa.repository.JpaRepository;

public interface CarRepository extends JpaRepository<Car, String> {
}
```

SellerDAO.java

```
package com.dao;

import com.bean.Car;
import com.bean.Seller;
import com.repository.CarRepository;
import com.repository.SellerRepository;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Repository;
import java.util.List;

@Repository
public class SellerDAO {

    @Autowired
    private SellerRepository sellerRepository;

    @Autowired
    private CarRepository carRepository;

    public void addSeller(Seller seller) {
        sellerRepository.save(seller);
    }

    public void buyCar(String sellerId, Car car) {
        Seller seller = sellerRepository.findById(sellerId).orElse(null);
        if (seller != null) {
            car.setSeller(seller);
            carRepository.save(car);
        }
    }

    public List<Seller> sellerWithMaximumSalesCount() {
```

```
        return sellerRepository.sellerWithMaximumSalesCount();
    }
}
```

SalesStatApplication.java

```
package com;

import org.springframework.boot.CommandLineRunner;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class SalesStatApplication implements CommandLineRunner {

    public static void main(String[] args) {
        SpringApplication.run(SalesStatApplication.class, args);
    }

    @Override
    public void run(String... args) throws Exception {
        System.out.println("Invoke the methods");
    }
}
```

04) Account with ATM Card With Logger

Pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>

<project xmlns="http://maven.apache.org/POM/4.0.0"

         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"

         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">

    <modelVersion>4.0.0</modelVersion>


    <parent>

        <groupId>org.springframework.boot</groupId>

        <artifactId>spring-boot-starter-parent</artifactId>
```

```
    <version>3.0.6</version>
    <relativePath/>
</parent>

<groupId>com</groupId>
<artifactId>BankAccountWithATMCard</artifactId>
<version>0.0.1-SNAPSHOT</version>
<name>BankAccountWithATMCard</name>
<description>BankAccountWithATMCard</description>

<properties>
    <java.version>17</java.version>
</properties>

<dependencies>
    <dependency>
        <groupId>org.springframework.restdocs</groupId>
        <artifactId>spring-restdocs-mockmvc</artifactId>
        <scope>test</scope>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-data-jpa</artifactId>
    </dependency>
    <dependency>
```

```
    <groupId>jakarta.persistence</groupId>
    <artifactId>jakarta.persistence-api</artifactId>
    <version>3.1.0</version>
</dependency>
<dependency>
    <groupId>org.slf4j</groupId>
    <artifactId>slf4j-api</artifactId>
    <version>1.7.36</version>
</dependency>
<dependency>
    <groupId>mysql</groupId>
    <artifactId>mysql-connector-java</artifactId>
    <version>8.0.26</version>
</dependency>
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-test</artifactId>
    <scope>test</scope>
</dependency>
<dependency>
    <groupId>junit</groupId>
    <artifactId>junit</artifactId>
    <version>4.12</version>
    <scope>test</scope>
</dependency>
<dependency>
    <groupId>ch.qos.logback</groupId>
    <artifactId>logback-classic</artifactId>
    <version>1.2.11</version>
```

```
        </dependency>
    </dependencies>

    <build>
        <plugins>
            <plugin>
                <groupId>org.springframework.boot</groupId>
                <artifactId>spring-boot-maven-plugin</artifactId>
            </plugin>
            <plugin>
                <groupId>org.codehaus.mojo</groupId>
                <artifactId>exec-maven-plugin</artifactId>
                <version>1.6.0</version>
                <configuration>
                    <mainClass>com.BankAccountWithATMCardApplication</mainClass>
                </configuration>
                <executions>
                    <execution>
                        <id>run-selenium</id>
                        <phase>integration-test</phase>
                        <goals>
                            <goal>java</goal>
                        </goals>
                    </execution>
                </executions>
            </plugin>
        </plugins>
    </build>
</project>
```

BankAccountWithATMCardApplication.java

```
package com;

import org.springframework.boot.CommandLineRunner;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class BankAccountWithATMCardApplication implements CommandLineRunner {

    public static void main(String[] args) {
        SpringApplication.run(BankAccountWithATMCardApplication.class, args);
    }

    @Override
    public void run(String... args) {
        System.out.println("Invoke the methods");
    }
}
```

Account.java

```
package com.bean;

import jakarta.persistence.*;

@Entity
@Inheritance(strategy = InheritanceType.SINGLE_TABLE)
@DiscriminatorColumn(name = "account_type", discriminatorType =
DiscriminatorType.STRING)
@Table(name = "account")
```

```
public abstract class Account {

    @Id
    private int accountNumber;
    private String holderName;
    private double balance;

    @OneToOne(mappedBy = "account", cascade = CascadeType.ALL)
    private ATMCard atmCard;

    public Account() {}

    public int getAccountNumber() {
        return accountNumber;
    }

    public void setAccountNumber(int accountNumber) {
        this.accountNumber = accountNumber;
    }

    public String getHolderName() {
        return holderName;
    }

    public void setHolderName(String holderName) {
        this.holderName = holderName;
    }

    public double getBalance() {
```

```
        return balance;
    }

    public void setBalance(double balance) {
        this.balance = balance;
    }

    public ATMCard getAtmCard() {
        return atmCard;
    }

    public void setAtmCard(ATMCard atmCard) {
        this.atmCard = atmCard;
    }
}
```

ATMCard.java

```
package com.bean;

import jakarta.persistence.*;
import java.util.Date;

@Entity
@Table(name = "atmcard")
public class ATMCard {

    @Id
    private String cardNumber;
    private String cardType;
```

```
private Date expiryDate;
private int cvvNumber;

@OneToOne
@JoinColumn(name = "account_number", unique = true)
private Account account;

public ATMCard() {}

public String getCardNumber() {
    return cardNumber;
}

public void setCardNumber(String cardNumber) {
    this.cardNumber = cardNumber;
}

public String getCardType() {
    return cardType;
}

public void setCardType(String cardType) {
    this.cardType = cardType;
}

public Date getExpiryDate() {
    return expiryDate;
}
```

```
    public void setExpiryDate(Date expiryDate) {
        this.expiryDate = expiryDate;
    }

    public int getCvvNumber() {
        return cvvNumber;
    }

    public void setCvvNumber(int cvvNumber) {
        this.cvvNumber = cvvNumber;
    }

    public Account getAccount() {
        return account;
    }

    public void setAccount(Account account) {
        this.account = account;
    }
}
```

CurrentAccount.java

```
package com.bean;

import jakarta.persistence.*;

@Entity
@DiscriminatorValue("CURR")
public class CurrentAccount extends Account {
```

```
    private double ODLimit;

    public CurrentAccount() {}

    public double getODLimit() {
        return ODLimit;
    }

    public void setODLimit(double ODLimit) {
        this.ODLimit = ODLimit;
    }
}
```

SavingAccount.java

```
package com.bean;

import jakarta.persistence.*;

@Entity
@DiscriminatorValue("SAV")
public class SavingsAccount extends Account {

    private double minimumBalance;

    public SavingsAccount() {}

    public double getMinimumBalance() {
        return minimumBalance;
```

```
    }

    public void setMinimumBalance(double minimumBalance) {
        this.minimumBalance = minimumBalance;
    }
}
```

AccountDAO.java

```
package com.dao;

import com.bean.Account;
import com.repository.AccountRepository;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Repository;

import java.util.List;

@Repository
public class AccountDAO {

    private static final Logger logger = LoggerFactory.getLogger(AccountDAO.class);

    @Autowired
    private AccountRepository accountRepository;

    public void openAccount(Account account) {
        accountRepository.save(account);
        logger.info("Account with id {} added successfully", account.getAccountNumber());
```

```
    }

    public List<Account> retrieveAccountBasedOnCardType(String cardType) {
        List<Account> accounts = accountRepository.findByAtmCardCardType(cardType);
        if (accounts.isEmpty()) {
            logger.error("No account with this card type {}", cardType);
        } else {
            logger.info("Account details with card type {} retrieved successfully", cardType);
        }
        return accounts;
    }
}
```

ATMCardDAO.java

```
package com.dao;

import com.bean.ATMCard;
import com.bean.Account;
import com.repository.ATMCardRepository;
import com.repository.AccountRepository;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Repository;

@Repository
public class ATMCardDAO {

    public static final Logger logger = LoggerFactory.getLogger(ATMCardDAO.class);
```

```
    @Autowired
    private AccountRepository accountRepository;

    @Autowired
    private ATMCardRepository atmCardRepository;

    public void issueATMCardToAccount(int accountNumber, ATMCard atmCardObject) {
        try {
            Account optionalAccount =
accountRepository.findById(accountNumber).orElse(null);
            if (optionalAccount != null && optionalAccount.getAtmCard() == null) {
                atmCardObject.setAccount(optionalAccount);
                optionalAccount.setAtmCard(atmCardObject);
                accountRepository.save(optionalAccount);
                logger.info("ATM card successfully issued to account number " + accountNumber);
            } else {
                logger.error("ATM card not issued to account number " + accountNumber);
            }
        } catch (Exception e) {
            logger.error("Error issuing ATM card to account number " + accountNumber + ": " +
e.getMessage());
        }
    }
}
```

AccountRepository.java

```
package com.repository;

import com.bean.Account;
import org.springframework.data.jpa.repository.JpaRepository;
```

```
import java.util.List;

public interface AccountRepository extends JpaRepository<Account, Integer> {
   List<Account> findByAtmCardCardType(String cardType);
}
```

ATMCardRepository.java

```
package com.repository;

import com.bean.ATMCard;
import org.springframework.data.jpa.repository.JpaRepository;

public interface ATMCardRepository extends JpaRepository<ATMCard, String> {
}
```

Logger.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<configuration>
   <property name="HOME_LOG" value="log/mylogs.log"/>

   <appender name="CONSOLE" class="ch.qos.logback.core.ConsoleAppender">
      <layout class="ch.qos.logback.classic.PatternLayout">
         <Pattern>%d{HH:mm:ss.SSS} [%t] %-5level %logger{36} - %msg%n</Pattern>
      </layout>
   </appender>

   <appender name="FILE" class="ch.qos.logback.core.FileAppender">
      <file>${HOME_LOG}</file>
      <append>true</append>
      <encoder>
```

```
        <pattern>[%-5level] %d{dd-MM-yyyy hh:mm:ss.SSS a} [%thread] %logger{50} -
%msg%n</pattern>
    </encoder>
  </appender>

  <!-- Only log to file for DAO and exception packages -->
  <logger name="com.dao" level="info" additivity="false">
    <appender-ref ref="FILE"/>
  </logger>

  <logger name="com.exception" level="info" additivity="false">
    <appender-ref ref="FILE"/>
  </logger>

  <!-- Root logger logs to console -->
  <root level="debug">
    <appender-ref ref="CONSOLE"/>
  </root>
</configuration>
```

05) Account Transaction

Pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
    xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
    xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <parent>
```

```
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.0.6</version>
    <relativePath/>
</parent>

<groupId>com</groupId>
<artifactId>AccountTransaction</artifactId>
<version>0.0.1-SNAPSHOT</version>
<name>AccountTransaction</name>
<description>AccountTransaction</description>

<properties>
    <java.version>17</java.version>
</properties>

<dependencies>
    <dependency>
        <groupId>org.springframework.restdocs</groupId>
        <artifactId>spring-restdocs-mockmvc</artifactId>
        <scope>test</scope>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-data-jpa</artifactId>
```

```
    </dependency>
    <dependency>
        <groupId>mysql</groupId>
        <artifactId>mysql-connector-java</artifactId>
        <version>8.0.26</version>
        <scope>runtime</scope>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-test</artifactId>
        <scope>test</scope>
    </dependency>
    <dependency>
        <groupId>junit</groupId>
        <artifactId>junit</artifactId>
        <version>4.12</version>
        <scope>test</scope>
    </dependency>
</dependencies>

<build>
    <plugins>
        <plugin>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-maven-plugin</artifactId>
        </plugin>
        <plugin>
            <groupId>org.codehaus.mojo</groupId>
            <artifactId>exec-maven-plugin</artifactId>
```

```
            <version>1.6.0</version>
            <configuration>
               <mainClass>com.AccountTransactionApplication</mainClass>
            </configuration>
            <executions>
               <execution>
                  <id>run-selenium</id>
                  <phase>integration-test</phase>
                  <goals>
                     <goal>java</goal>
                  </goals>
               </execution>
            </executions>
         </plugin>
      </plugins>
   </build>
</project>
```

AccountTransaction.java

```
package com;

import org.springframework.boot.CommandLineRunner;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class AccountTransactionApplication implements CommandLineRunner {

   public static void main(String[] args) {
```

```
        SpringApplication.run(AccountTransactionApplication.class, args);
    }

    @Override
    public void run(String... args) {
        System.out.println("Invoke the methods");
    }
}
```

Account.java

```
package com.bean;

import jakarta.persistence.*;
import java.time.LocalDate;
import java.util.ArrayList;
import java.util.List;

@Entity
@Table(name = "account")
public class Account {

    @Id
    private int accountNumber;
    private String holderName;
    private LocalDate accountDate;
    private double balance;

    @OneToMany(mappedBy = "account", cascade = CascadeType.ALL)
    private List<Transactions> transactionList = new ArrayList<>();
```

```
    // Getters and Setters
    public int getAccountNumber() { return accountNumber; }
    public void setAccountNumber(int accountNumber) { this.accountNumber =
accountNumber; }

    public String getHolderName() { return holderName; }
    public void setHolderName(String holderName) { this.holderName = holderName; }

    public LocalDate getAccountDate() { return accountDate; }
    public void setAccountDate(LocalDate accountDate) { this.accountDate = accountDate; }

    public double getBalance() { return balance; }
    public void setBalance(double balance) { this.balance = balance; }

    public List<Transactions> getTransactionList() { return transactionList; }
    public void setTransactionList(List<Transactions> transactionList) { this.transactionList =
transactionList; }
}
```

Transactions.java

```
package com.bean;

import jakarta.persistence.*;
import java.time.LocalDate;

@Entity
@Table(name = "transactions")
public class Transactions {

    @Id
```

```
    private int transactionId;
    private LocalDate transactionDate;
    private String transactionType;
    private double amount;

    @ManyToOne
    @JoinColumn(name = "accountnumber")
    private Account account;

    // Getters and Setters
    public int getTransactionId() { return transactionId; }
    public void setTransactionId(int transactionId) { this.transactionId = transactionId; }

    public LocalDate getTransactionDate() { return transactionDate; }
    public void setTransactionDate(LocalDate transactionDate) { this.transactionDate =
transactionDate; }

    public String getTransactionType() { return transactionType; }
    public void setTransactionType(String transactionType) { this.transactionType =
transactionType; }

    public double getAmount() { return amount; }
    public void setAmount(double amount) { this.amount = amount; }

    public Account getAccount() { return account; }
    public void setAccount(Account account) { this.account = account; }
}
```

AccounbtDAO.java

```
package com.dao;
```

```
import java.time.LocalDate;
import java.util.List;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Repository;

import com.bean.Account;
import com.bean.Transactions;
import com.repository.AccountRepository;
import com.repository.TransactionsRepository;

@Repository
public class AccountDAO {

    @Autowired
    private AccountRepository accountRepository;

    @Autowired
    private TransactionsRepository transactionsRepository;

    public void addAccount(Account account) {
        accountRepository.save(account);
    }

    public void performTransactionOnAccount(int accountNumber, Transactions transaction) {
        Account account = accountRepository.findById(accountNumber).orElse(null);
        if (account != null) {
            transaction.setAccount(account);
```

```
            account.getTransactionList().add(transaction);
            accountRepository.save(account); // saves both account and transaction due to cascade
        }
    }

    public List<Transactions> retrieveTransactionDetails(int accountNumber, LocalDate startDate, LocalDate endDate) {
        return transactionsRepository.findByAccountAccountNumberAndTransactionDateBetween(accountNumber, startDate, endDate);
    }
}
```

AccountRepository.java

```
package com.repository;

import org.springframework.data.jpa.repository.JpaRepository;
import com.bean.Account;

public interface AccountRepository extends JpaRepository<Account, Integer> {
}
```

TransactionsRepository.java

```
package com.repository;

import org.springframework.data.jpa.repository.JpaRepository;
import com.bean.Transactions;
import java.time.LocalDate;
import java.util.List;

public interface TransactionsRepository extends JpaRepository<Transactions, Integer> {
```

```
    List<Transactions> findByAccountAccountNumberAndTransactionDateBetween(int
accountNumber, LocalDate startDate, LocalDate endDate);

}
```

ApplicationProperties

```
spring.application.name=AccountTransaction

server.port=8090


spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver

spring.datasource.url=jdbc:mysql://localhost:3306/test23

spring.datasource.username=root

spring.datasource.password=@Ibm12345


spring.jpa.generate-ddl=true

spring.jpa.hibernate.ddl-auto=create

spring.jpa.show-sql=true

# spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQL8Dialect
```